



KAPITEL

9

Fehler vermeiden

Lernziele

Zentrale Konzepte in diesem Kapitel: Testen, Modultests, Fehler beseitigen, Tests automatisieren

Java-Konstrukte in diesem Kapitel: (In diesem Kapitel werden keine neuen Java-Konstrukte vorgestellt.)

9.1 Einführung

Wenn Sie die vorherigen Kapitel gelesen und die vorgeschlagenen Übungen implementiert haben, dann haben Sie inzwischen etliche Klassen geschrieben. Möglicherweise haben Sie dabei festgestellt, dass der erste Entwurf einer Klasse selten perfekt ist. Normalerweise ist der Quelltext anfangs nicht vollständig, sodass meist noch einiges getan werden muss, um die Implementierung abzuschließen.

Die Probleme, die Sie dabei haben, werden sich über die Zeit verschieben. Programmieranfänger kämpfen typischerweise mit *Syntaxfehlern*. Syntaxfehler sind die Fehler, die in der Struktur des Quelltextes liegen. Sie sind leicht zu finden, weil der Compiler sie entdeckt und eine Fehlermeldung ausgibt.

Erfahrene Programmierer, die kompliziertere Aufgaben angehen, haben üblicherweise weniger Probleme mit der Syntax einer Sprache. Sie beschäftigen sich stattdessen eher mit *logischen Fehlern*.

Ein logischer Fehler liegt vor, wenn ein Programm ohne Probleme übersetzt und ausgeführt werden kann, aber nicht das gewünschte Ergebnis liefert. Logische Probleme sind sehr viel schwieriger zu finden als Syntaxfehler. Tatsächlich ist oft kaum zu erkennen, dass ein Fehler vorliegt.

Syntaktisch korrekte Programme zu schreiben ist relativ leicht zu erlernen, denn es gibt gute Werkzeuge (wie Compiler), die Syntaxfehler entdecken und anzeigen. Logisch korrekte Programme zu schreiben ist hingegen sehr schwierig für nahezu jedes nichttriviale Problem, und der Beweis für ihre Korrektheit kann für den allgemeinen Fall nicht automatisiert werden. Es ist tatsächlich so schwierig, dass der Großteil der kommerziell vertriebenen Software bekanntermaßen eine signifikante Anzahl von Fehlern enthält.

Somit ist es für einen kompetenten Softwaretechniker unverzichtbar, Probleme im Zusammenhang mit Korrektheit zu verstehen und die Anzahl der Fehler in einer Klasse reduzieren zu können.

Konzept

Testen dient der Überprüfung, ob ein Stück Software (eine Methode, eine Klasse oder ein Programm) das gewünschte Verhalten zeigt.

In diesem Kapitel werden wir eine Reihe von Praktiken vorstellen, mit denen die Korrektheit von Software verbessert werden kann. Dazu gehören Testen, Fehlerbeseitigung und die Erstellung wartbarer Quelltexte.

Mit *Testen* wird herausgefunden, ob ein Stück Software Fehler enthält. Gutes Testen ist nicht einfach, denn es gibt sehr viel dabei zu bedenken.

Die *Fehlerbeseitigung* kommt nach dem Testen: Wenn Fehler durch Tests aufgezeigt wurden, verwenden wir Techniken zur Fehlerbeseitigung, um die Ursache dieser Fehler zu entdecken und sie zu beseitigen. Dies kann erheblichen Aufwand bedeuten, denn zwischen dem Wissen, dass ein Fehler existiert, dem Finden der Ursache und dem Beheben ist es oft ein weiter Weg.

Konzept

Fehlerbeseitigung bezeichnet die Suche nach der Ursache eines Fehlers und seine Beseitigung.

Die *Erstellung wartbarer Quelltexte* ist vermutlich der grundlegendste Ansatz. Es gilt, Quelltexte so zu schreiben, dass Fehler von vornherein vermieden werden oder zumindest, wenn sie sich doch einschleichen, leicht zu finden sind. Der Programmierstil und die Kommentierung haben hierauf einen starken Einfluss. Idealerweise sollte Quelltext so einfach zu verstehen sein, dass der ursprüngliche Entwickler Fehler vermeidet und der Wartungsprogrammierer mögliche Fehler leicht entdecken kann.

In der Praxis ist das alles nicht immer so einfach. Aber auf jeden Fall gibt es deutliche Unterschiede zwischen gut und schlecht geschriebenen Programmen, sowohl in der Anzahl der Fehler als auch dem Aufwand ihrer Beseitigung.

9.2 Testen und Fehlerbeseitigung

Testen und Fehlerbeseitigung sind grundlegende Tätigkeiten bei der Softwareentwicklung. Sie werden häufig Ihre Programme auf Fehler untersuchen und dann gegebenenfalls die Fehlerursache feststellen wollen. Außerdem werden Sie möglicherweise auch für das Testen oder Anpassen der Programme anderer zuständig sein. Im letzteren Fall ist die Fehlerbeseitigung oft eng verknüpft mit dem Verstehen der Quelltexte anderer, sodass die Techniken für beides sich stark überschneiden. In den nächsten Abschnitten werden wir folgende Techniken und Werkzeuge zum Testen und zur Fehlerbeseitigung untersuchen:

- manuelle Modultests mit BlueJ
- automatisierte Tests
- manuelle Ausführung
- **print**-Anweisungen
- Debugger

Wir werden die ersten beiden Testtechniken im Zusammenhang mit Klassen untersuchen, die Sie selbst geschrieben haben könnten, und die übrigen Techniken zur Fehlerbeseitigung in einem Kontext, in dem Sie fremde Quelltexte verstehen müssen.

9.3 Modultests in BlueJ

Der Begriff *Modultest* (*unit test*) bezieht sich auf Tests einzelner Einheiten einer Anwendung, im Gegensatz zu *Akzeptanztests*, die eine ganze Anwendung testen. Die Größe einer solchen Einheit kann stark variieren. Es kann eine Gruppe von Klassen sein, eine einzelne Klasse oder sogar nur eine einzelne Methode. Modultests können lange vor der Fertigstellung einer Anwendung durchgeführt werden. Jede einzelne Methode kann (und sollte), sobald sie geschrieben und übersetzt wurde, getestet werden.

Weil BlueJ uns den interaktiven Umgang mit individuellen Objekten ermöglicht, gibt es uns gute Möglichkeiten für das Testen von Klassen und Methoden. Beachten Sie auch, dass es nie zu früh zum Testen ist. Frühes Experimentieren und Testen hat viele Vorteile. Erstens macht es das Finden und Beheben von Fehlern in einem System einfacher und billiger, als wenn sie erst sehr viel später im Laufe der Systementwicklung gefunden werden. Zweitens können wir auf diese Weise eine Reihe von Testfällen und Ergebnissen sammeln, die wir immer wieder einsetzen können, während das System wächst. Jedes Mal, wenn wir eine Änderung an dem System vornehmen, erlauben uns diese Testfälle zu prüfen, ob wir mit diesen Änderungen ungewollt Fehler eingeführt haben.

Um diese Art des Testens zu demonstrieren, werden wir das Projekt *OnlineShop* verwenden. Es repräsentiert den frühen Status bei der Entwicklung einer Software für einen elektronischen Onlineshop (wie Amazon.com). Unser Projekt umfasst nur einen sehr kleinen Teil dieser Anwendung, und zwar den, der sich mit den Kundenkommentaren zu Verkaufsartikeln befasst.

Öffnen Sie das Projekt *OnlineShop*. Es enthält in diesem Stadium zwei Klassen: **Produkt** und **Kommentar**. Folgende Funktionen sind in diesem Teil der Anwendung enthalten, wenn wir uns ausschließlich auf die Bearbeitung der Kundenkommentare konzentrieren:

- Es können Produkte mit einer Beschreibung und einem Preis erzeugt werden.
- Es können Kundenkommentare zu den Produkten hinzugefügt und entfernt werden.
- Kommentare bestehen aus einem Kommentartext, dem Namen des Autors und einer Bewertung. Die Bewertung liegt im Bereich 1 bis 5 (inklusive).
- Jede Person kann nur einen Kommentar abgeben. Nachfolgende Versuche, einen erneuten Kommentar abzugeben, werden abgeblockt.
- Die Benutzungsschnittstelle (die in diesem Projekt noch nicht implementiert ist) umfasst die Frage „War dieser Kommentar hilfreich?“ einschließlich der Knöpfe JA und NEIN. Durch Anklicken von JA oder NEIN stimmt der Benutzer dem Kommentar zu oder lehnt ihn ab. Das Verhältnis der Zustimmungen und Ablehnungen wird für die Kommentare gespeichert, sodass die nützlichsten Kommentare (die mit dem höchsten Stimmenverhältnis) ganz oben angezeigt werden können.

Die Klasse **Kommentar** in unserem Projekt speichert Informationen über einen einzelnen Kommentar. Zum Testen konzentrieren wir uns auf die Klasse **Produkt** aus Listing 9.1. Objekte dieser Klasse repräsentieren jeweils ein Produkt, einschließlich aller dafür abgegebenen Kommentare.

Konzept

Modultests

beziehen sich auf Tests einzelner Teile einer Anwendung, zum Beispiel auf Methoden und Klassen.

Zum Testen gehört, dass wir mehrere Teile der beabsichtigten Funktionen prüfen, einschließlich:

- Können Kommentare zu einem Produkt hinzugefügt oder entfernt werden?
- Zeigt die Methode **infoZeigen** korrekt alle Informationen zu einem Produkt an?
- Werden die Beschränkungen (Bewertungen müssen zwischen 1 und 5 liegen, nur ein Kommentar pro Person) korrekt umgesetzt?
- Können wir korrekt den hilfreichsten Kommentar finden (den mit den meisten Stimmen)?

Wir werden feststellen, dass wir all dies komfortabel mithilfe der Objektleiste von BlueJ testen können. Außerdem werden wir feststellen, dass BlueJ einige Testabläufe vereinfacht, da gezielt Änderungen an den zu testenden Klassen vorgenommen werden können.

Listing 9.1

Die Klasse **Produkt**.

```
import java.util.ArrayList;
import java.util.Iterator;

/**
 * Diese Klasse repräsentiert Produkte einer Online-E-Commerce-Website (wie
 * z.B. Amazon.com). Produkt-Objekte speichern alle für das Produkt relevanten Daten,
 * einschließlich einer Beschreibung, dem Preis, Kommentare der Kunden etc.
 *
 * HINWEIS: Die vorliegende Version ist nicht vollständig! Derzeit ist nur der
 * Code für die Behandlung der Kundenkommentare vorhanden.
 *
 * @autor Michael Kölling und David J. Barnes
 * @version 0.1 (2016.02.29)
 */
public class Produkt
{
    private String name;
    private int preis; // in Cent
    private ArrayList<Kommentar> kommentare;

    /**
     * Erzeuge ein neues Produkt.
     */
    public Produkt(String name, int preis)
    {
        this.name = name;
        this.preis = preis;
        kommentare = new ArrayList<>();
    }

    /**
     * Liefere den Namen des Produkts.
     */
    public String gibName()
    {
        return name;
    }

    /**
     * Liefere den Preis des Produkts.
     */
    public int gibPreis()
    {
        return preis;
    }

    /**
     * Liefere die Anzahl an Kundenkommentaren zu diesem Produkt.
     */
    public int gibAnzahlKommentare()
    {
        return kommentare.size();
    }
}
```

```

/**
 * Füge der Kommentarlste dieses Produkts einen Kommentar hinzu. Liefere im
 * Erfolgsfall true, und false, wenn der Kommentar abgelehnt wurde.
 *
 * Der Kommentar wird abgelehnt, wenn der Autor bereits einen Kommentar
 * hinterlassen hat oder wenn die Bewertung ungültig ist. Gültige Bewertungen
 * liegen zwischen 1 und 5 (inklusive).
 */
public boolean kommentarHinzufuegen(String autor, String text, int bewertung)
{
    if(bewertungUnguelteig(bewertung)) { // ungültige Bewertungen ablehnen
        return false;
    }

    if(findeKommentar(autor) != null) { // mehrere Bewertungen desselben
        // Autors ablehnen
        return false;
    }

    kommentare.add(new Kommentar(autor, text, bewertung));
    return true;
}

/**
 * Lösche den Kommentar, der unter dem angegebenen Index gespeichert ist. Ist der
 * Index ungültig, tue nichts.
 */
public void kommentarLoeschen(int index)
{
    if(index >=0 && index < kommentare.size()) { // wenn der Index gültig ist
        kommentare.remove(index);
    }
}

/**
 * Stimme dem Kommentar unter 'index' zu. Das heißt, stufe diesen Kommentar als
 * hilfreich ein. Ist der Index ungültig, tue nichts.
 */
public void kommentarZustimmen(int index)
{
    if(index >=0 && index < kommentare.size()) { // wenn der Index gültig ist
        kommentare.get(index).zustimmen();
    }
}

/**
 * Lehne den Kommentar unter 'index' ab. Das heißt, stufe diesen Kommentar als
 * weniger hilfreich ein. Ist der Index ungültig, tue nichts.
 */
public void kommentarAblehnen(int index)
{
    if(index >=0 && index < kommentare.size()) { // wenn der Index gültig ist
        kommentare.get(index).ablehnen();
    }
}

/**
 * Gib alle Kommentare aus. (Zum Testen erfolgt die Ausgabe derzeit auf die
 * Konsole. Soll später überarbeitet werden, sodass die Ausgabe im Web erfolgt.)
 */
public void infoZeigen()
{
    System.out.println("**** " + name + " ****");
    System.out.println("Preis: " + preisString(preis));
    System.out.println();
    System.out.println("Kundenkommentare:");
    for(Kommentar kommentar : kommentare) {
        System.out.println("-----");
        System.out.println(kommentar.gibAlleDetails());
    }
    System.out.println();
    System.out.println("=====");
}

```

```

/**
 * Liefere den Kommentar, der am hilfreichsten ist. Der nützlichste
 * Kommentar ist derjenige mit dem höchsten Stimmenverhältnis. Gibt es mehrere
 * Kommentare mit dem höchsten Stimmenverhältnis, liefere irgendeinen von ihnen.
 */
public Kommentar findeHilfreichstenKommentar()
{
    Iterator<Kommentar> it = kommentare.iterator();
    Kommentar bester = it.next();
    while(it.hasNext())
    {
        Kommentar aktuell = it.next();
        if(aktuell.gibStimmenAnzahl() > bester.gibStimmenAnzahl()) {
            bester = aktuell;
        }
    }
    return bester;
}

/**
 * Prüfe, ob die gegebene Bewertung ungültig ist. Liefere true, wenn
 * die Bewertung ungültig ist. Gültige Bewertungen liegen
 * im Bereich [1..5].
 */
private boolean bewertungUngueltig(int bewertung)
{
    return bewertung < 0 || bewertung > 5;
}

/**
 * Finde den Kommentar des angegebenen Autors.
 * @return der Kommentar, falls ein solcher existiert, ansonsten null.
 */
private Kommentar findeKommentar(String autor)
{
    for(Kommentar kommentar : kommentare) {
        if(kommentar.gibAutor().equals(autor)) {
            return kommentar;
        }
    }
    return null;
}

/**
 * Liefere für einen als int angegebenen Preis den zugehörigen String, der
 * denselben Preis repräsentiert. Der Preis wird in ganzen Cent angegeben.
 * Für preis==12345 wird z.B. der folgende String zurückgeliefert:
 * Euro 123,45
 */
private String preisString(int preis)
{
    int euro = preis / 100;
    int cents = preis - (euro*100);
    if(cents <= 9) {
        return "Euro" + euro + ",0" + cents; // falls nötig 0 einfügen
    }
    else {
        return "Euro" + euro + "," + cents;
    }
}

```

9.3.1 Objektinspektoren nutzen

Beim interaktiven Testen ist der Einsatz von Objektinspektoren oft sehr hilfreich. Als Vorbereitung für die Tests erzeugen Sie ein **Produkt**-Objekt auf der Objektleiste und öffnen seinen Objektinspektor, indem Sie die Funktion **INSPIZIEREN** aus dem Kontextmenü des Objekts auswählen. Selektieren Sie das Datenfeld **kommentare** und öffnen Sie auch dessen Inspektor (Abbildung 9.1). Prüfen Sie, ob die Liste erzeugt wurde (nicht **null** ist) ist und am Anfang die Größe 0 hat. Prüfen Sie auch, dass die Liste

beim Hinzufügen von Kommentaren anwächst. Lassen Sie den Kommentarlisten-Inspektor als Hilfe für die folgenden Tests geöffnet.

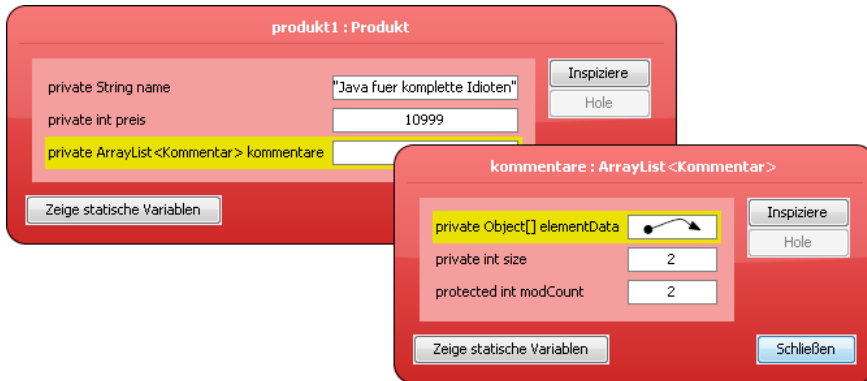


Abbildung 9.1
Objektinspektor für die
Liste **kommentare**.

Ein wichtiger Fall beim Testen von Klassen, die Datenstrukturen benutzen, ist die Überprüfung, ob sie sich bei einer leeren und bei einer vollständig gefüllten Datenstruktur korrekt verhalten. Das Testen voller Datenstrukturen ist nur auf solche Strukturen anzuwenden, die eine Obergrenze angegeben haben, wie beispielsweise Arrays. In unserem Fall verwenden wir eine **ArrayList**, sodass nicht geprüft werden kann, ob die Liste voll ist, weil die Liste nach Bedarf erweitert werden kann. Eine leere Liste zu testen, ist wichtig, da dies einen Sonderfall darstellt, der besonders behandelt wird.

Der erste Test, der auf **Produkt** ausgeführt werden kann, besteht darin, ihre Methode **infoZeigen** aufzurufen, bevor Kommentare hinzugefügt werden. Diese sollte korrekt die Beschreibung und den Preis des Produkts ohne irgendwelche Kommentare anzeigen.

Ein Schlüsselement beim Testen ist das Überprüfen von *Grenzfällen*, weil gerade dort häufig etwas schiefgeht. Ein Grenzfall für die Klasse **Produkt** ist beispielsweise die leere Kommentarliste. Die Grenzfälle für die Klasse **Kommentar** ergeben sich z.B. durch die Beschränkung der Bewertung auf den Bereich 1 bis 5. Bewertungen ganz oben und unten in diesem Bereich sind Grenzfälle. Es ist wichtig, nicht nur die Bewertungen in der Mitte dieses Bereichs zu prüfen, sondern auch die höchst- und kleinstmögliche Bewertung.

Um Tests nach diesen Überlegungen durchführen zu können, erzeugen Sie ein **Produkt**-Objekt auf der Objektleiste und führen Sie die folgenden Übungen als erste Tests der Kommentarfunktionen durch. Wenn Sie sorgfältig testen, sollten Sie zwei Fehler in Ihrem Quelltext finden.

Übung 9.1 Fügen Sie mehrere Kommentare zu den Produkten hinzu, während Sie den Inspektor für die Kommentarliste im Blick haben. Achten Sie darauf, dass sich die Liste wie erwartet verhält (d.h., sie sollte länger werden). Vielleicht wollen Sie auch das Datenfeld **elementData** des **ArrayList**-Objekts untersuchen.

Übung 9.2 Überprüfen Sie, ob die Methode `infoZeigen` die Produktinformation, einschließlich der Kommentare, korrekt anzeigt. Versuchen Sie es sowohl für Elemente mit und ohne Kommentare.

Übung 9.3 Prüfen Sie, ob die Methode `gibAnzahlKommentare` wie erwartet funktioniert.

Übung 9.4 Prüfen Sie anschließend, ob Zweifach-Autoren korrekt behandelt werden – d.h., dass weitere Kommentare des gleichen Autors abgelehnt werden. Wenn versucht wird, einen Kommentar eines Autors hinzuzufügen, von dem bereits ein Kommentar existiert, sollte die Methode `kommentarHinzufuegen` den booleschen Wert `false` zurückliefern. Prüfen Sie ebenfalls, dass der Kommentar nicht der Liste hinzugefügt wurde.

Übung 9.5 Prüfen Sie die Grenzfälle der Bewertungswerte. Das heißt, erzeugen Sie nicht nur Kommentare mit mittlerer Bewertung, sondern auch mit sehr hohen und sehr niedrigen Bewertungen. Funktioniert alles korrekt?

Übung 9.6 Gutes Testen der Grenzfälle schließt auch Tests ein, die bewusst außerhalb des gültigen Bereichs liegen. Testen Sie 0 und 6 als Bewertungswerte. In beiden Fällen sollte der Kommentar abgelehnt werden (`kommentarHinzufuegen` sollte `false` zurückliefern und der Kommentar sollte nicht zu der Kommentarliste hinzugefügt werden).

Übung 9.7 Testen Sie die Methoden `kommentarZustimmen` und `kommentarAblehnen`. Stellen Sie sicher, dass das Stimmenverhältnis korrekt gezählt wird.

Übung 9.8 Verwenden Sie die Methoden `kommentarZustimmen` und `kommentarAblehnen`, um einige Kommentare als sehr oder weniger hilfreich zu markieren. Testen Sie daraufhin die Methode `findeHilfreichstenKommentar`. Diese Methode sollte den Kommentar zurückliefern, der als am hilfreichsten gewählt wurde. Sie werden feststellen, dass die Methode eine Objektreferenz zurückliefert. Mit der `INSPIZIERE`-Funktion in dem Dialogfenster *Methodenergebnisse* können Sie prüfen, ob der korrekte Kommentar zurückgeliefert wurde. Natürlich müssen Sie wissen, welches der korrekte Kommentar ist, um zu prüfen, ob Sie das richtige Ergebnis haben!

Übung 9.9 Führen Sie Grenzbereichstests für die Methode `findeHilfreichstenKommentar` aus. Das heißt, rufen Sie diese Methode auf, wenn die Kommentarliste leer ist (noch keine Kommentare hinzugefügt wurden). Funktioniert dies wie erwartet?

Übung 9.10 Die Tests in den vorherigen Übungen sollten zwei Fehler in unserem Quelltext ans Tageslicht bringen. Beheben Sie sie. Können Sie anschließend mit Sicherheit annehmen, dass alle vorherigen Tests wie zuvor funktionieren? In Abschnitt 9.4 werden einige Probleme beim Testen diskutiert, die sich durch das Korrigieren oder Erweitern von Software ergeben.

Anhand dieser Übungen ist zu sehen, dass interaktive Methodenaufrufe und die Objektinspektoren nützlich sind, um unmittelbar Feedback über den Zustand eines Objekts zu bekommen. Sie ersparen häufig den Einsatz von **print**-Anweisungen (Abschnitt 9.9) beim Testen und der Fehlerbeseitigung.

9.3.2 Positives und negatives Testen

Wenn wir Tests planen, unterscheiden wir häufig *positive* und *negative* Testfälle. Positives Testen überprüft die Funktionalität, die wir erwarten. Das Hinzufügen eines Kommentars von einem neuen Autor mit einer gültigen Bewertung ist beispielsweise ein positiver Test. Bei der Überprüfung positiver Testfälle müssen wir uns davon überzeugen, dass die getestete Einheit auch wirklich das tut, was wir erwarten.

Negatives Testen überprüft die Fälle, deren Scheitern wir erwarten. Die Verwendung einer ungültigen Bewertung oder der Versuch, einen zweiten Kommentar vom gleichen Autor zu speichern, sind beides Beispiele für negative Tests. Wenn wir negative Testfälle überprüfen, erwarten wir, dass das Programm diese in kontrollierter Weise handhabt.

Konzept

Positives Testen ist das Testen der Fälle, die funktionieren sollten.

Negatives Testen ist das Testen der Fälle, die fehlschlagen sollten.

Fallstrick

Ein von Anfängern häufig gemachter Fehler ist es, ausschließlich positive Tests durchzuführen. Negatives Testen – das Prüfen, ob die Fälle, die fehlschlagen sollten, auch tatsächlich fehlschlagen, und das in möglichst organisierter Weise – ist ein wichtiger Bestandteil jedes guten Testvorgehens.

Übung 9.11 Welche der in den vorigen Übungen vorgeschlagenen Testfälle sind positive, welche sind negative Testfälle? Erstellen Sie eine Tabelle für jede Kategorie. Fallen Ihnen weitere positive Tests ein? Gibt es weitere negative Tests?

9.4 Tests automatisieren

Gründliches Testen wird unter anderem deshalb häufig vernachlässigt, weil Testen eine zeitaufwendige und relativ langweilige Tätigkeit ist, wenn sie von Hand ausgeführt wird. Ihnen ist bei den Übungen des vorherigen Abschnitts sicherlich aufgefallen, dass gründliches Testen sehr schnell lästig werden kann. Das bekommt insbesondere dann Gewicht, wenn Tests nicht nur einmal, sondern möglicherweise hundertmal oder tausendmal ausgeführt werden müssen. Glücklicherweise gibt es Techniken, die das wiederholte Testen automatisieren und dem Testen somit vieles von seiner Eintönigkeit nehmen. Im nächsten Abschnitt betrachten wir Testautomatisierung im Zusammenhang mit *Regressionstests*.

Konzept

Automatisieren von Tests vereinfacht den Prozess des Regressionstestens.

9.4.1 Regressionstests

Es wäre schön, wenn wir uns darauf verlassen könnten, dass durch das Korrigieren von Fehlern die Qualität eines Programms verbessert wird. Leider zeigt die Erfahrung, dass beim Modifizieren von Software nur allzu leicht weitere Fehler gemacht werden. Das Beheben eines Fehlers an einer Stelle kann leicht einen neuen Fehler an einer anderen Stelle mit sich führen.

Deshalb ist es wünschenswert, dass wir nach jeder Änderung an einer Software sogenannte *Regressionstests* durchführen. Ein Regressionstest ist ein Test, der bereits erfolgreich gelaufen ist und wiederholt wird, um eine neue Version der Software zu überprüfen. Die Wahrscheinlichkeit, dass Regressionstests wirklich durchgeführt werden, steigt, wenn sie in geeigneter Weise automatisiert werden können. Einer der einfachsten Wege, einen Regressionstest zu automatisieren, ist das Schreiben von Programmen, die als *Testgerüste* dienen können.

9.4.2 Automatisierte Tests mit JUnit

BlueJ sowie viele andere Entwicklungsumgebungen unterstützen Regressionstests durch ein Testsystem namens JUnit. JUnit ist ein von Erich Gamma und Kent Beck entwickeltes Test-Rahmenwerk für Java, doch inzwischen gibt es ähnliche Systeme für viele weitere Programmiersprachen.

JUnit ist ein populäres Test-Rahmenwerk zur Unterstützung von wohlstrukturierten Modultests und Regressionstests in Java. Es ist unabhängig von spezifischen Entwicklungsumgebungen erhältlich, aber auch in vielen Umgebungen bereits integriert. JUnit wurde von Erich Gamma und Kent Beck entwickelt. Sie können die Software selbst und eine Menge Informationen darüber finden unter <http://www.junit.org>.

Um an einem eigenen Beispiel zu sehen, wie Regressionstests durchgeführt werden, öffnen Sie das Projekt *OnlineShop-JUnit*. Dieses Projekt enthält die gleichen Klassen wie das vorherige Beispiel, plus einer zusätzlichen Klasse **ProduktTest**.

ProduktTest ist eine Testklasse. Das erste, was an dieser Klasse auffällt, ist, dass ihr Erscheinungsbild von denen der bisherigen Klassen abweicht (Abbildung 9.2). Sie ist mit dem Zusatz `<<unit test>>` versehen, sie unterscheidet sich farblich von den gewöhnlichen Klassen im Diagramm und sie ist mit der Klasse **Produkt** verbunden (sie wird mit dieser Klasse verschoben, wenn die Klasse **Produkt** im Diagramm verschoben werden sollte).

Auch dürfte Ihnen in Abbildung 9.2 aufgefallen sein, dass im Hauptfenster unter dem ÜBERSETZEN-Knopf einige weitere Steuerelemente zu finden sind. Diese erlauben Ihnen, von den integrierten Regressionstestwerkzeugen Gebrauch zu machen. Wenn Sie JUnit noch nicht in BlueJ verwendet haben, sind diese Werkzeuge deaktiviert und die Knöpfe sind in Ihrem System noch nicht zu sehen. Sie sollten diese Testwerkzeuge jetzt aktivieren. Öffnen Sie dazu im Dialogfeld EINSTELLUNGEN die Registerseite INTERFACE und stellen Sie sicher, dass die Option TESTWERKZEUGE ANZEIGEN markiert ist.

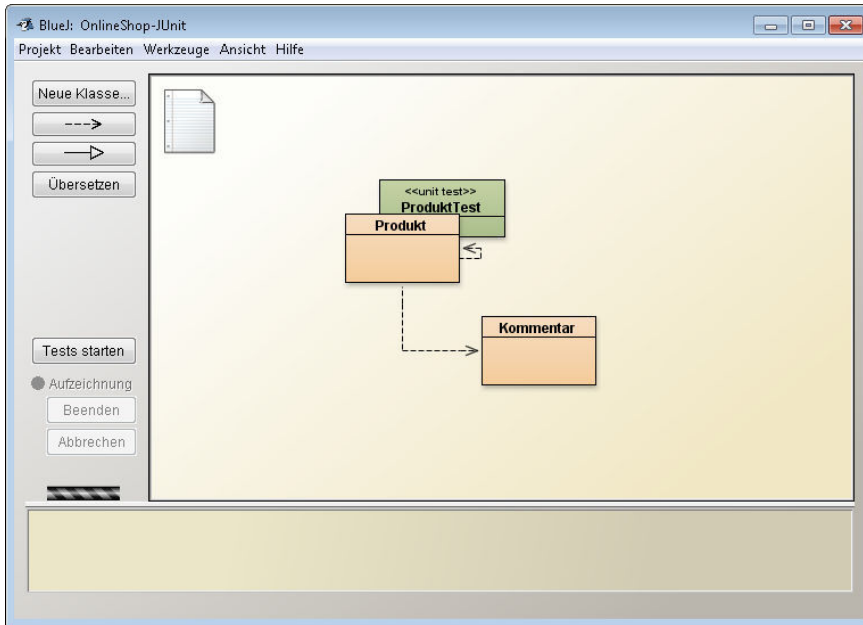


Abbildung 9.2
Ein Projekt mit
einer Testklasse.

Ein weiterer Unterschied zeigt sich im Kontextmenü, wenn Sie die Testklasse mit der rechten Maustaste anklicken (Abbildung 9.3). Es gibt drei neue Abschnitte in diesem Menü anstelle einer Liste von Konstruktoren.

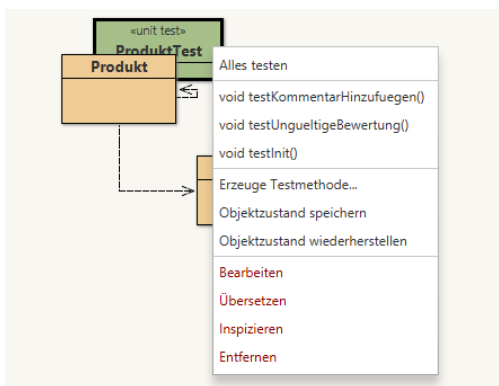


Abbildung 9.3
Das Kontextmenü für
eine Testklasse.

Mithilfe von Testklassen können wir unsere Regressionstests automatisieren. Die Testklasse enthält Quelltext, um eine Reihe von vorbereiteten Tests durchzuführen und ihre Ergebnisse zu prüfen. Dadurch wird das häufig wiederholte Testen mit den gleichen Tests stark vereinfacht.

Eine Testklasse ist normalerweise einer normalen Projektklasse zugeordnet. In diesem Fall gehört die Klasse **ProduktTest** zu der Klasse **Produkt** und wir sagen, dass **Produkt** die Bezugsklasse für **ProduktTest** ist.

In unserem Projekt wurde die Testklasse zuvor erzeugt; sie enthält bereits einige Tests. Wir können diese Tests ausführen, indem wir auf den Knopf **TESTS STARTEN** klicken.

Übung 9.12 Klicken Sie auf den Knopf TESTS STARTEN, um die Tests für Ihr Projekt auszuführen. Sie sollten ein Fenster wie in Abbildung 9.4 erhalten, das die Ergebnisse der Tests zusammenfasst.

Abbildung 9.4
Das Fenster
Testergebnisse.

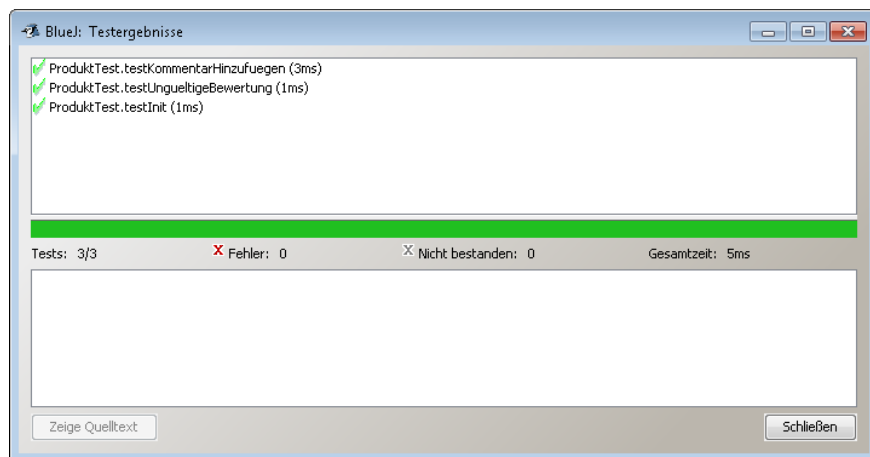


Abbildung 9.4 zeigt das Ergebnis der Ausführung von drei Tests, die **testKommentarHinzufuegen**, **testInit** und **testUnguelteBewertung** heißen und in der Testklasse definiert sind. Der Haken links neben den Namen der Tests zeigt an, dass die Tests erfolgreich waren. Sie erhalten die gleichen Ergebnisse, wenn Sie die Option ALLES TESTEN aus dem Kontextmenü der Textklasse aufrufen oder die Tests einzeln aus dem Kontextmenü auswählen.

Testklassen unterscheiden sich deutlich von gewöhnlichen Klassen, und wenn Sie sich den Quelltext der Klasse **ProduktTest** ansehen, dann werden Sie einige neue Dinge bemerken. An dieser Stelle des Buches wollen wir nicht im Detail erklären, wie Testklassen funktionieren, aber es sollte erwähnt werden, dass der Quelltext von **ProduktTest**, obwohl er von einer Person geschrieben sein könnte, tatsächlich innerhalb von BlueJ *automatisch generiert* wurde. Anschließend wurden lediglich einige Kommentare eingefügt, um den Zweck der Tests zu dokumentieren.

Jede Testklasse enthält normalerweise Tests zu den Funktionen ihrer Bezugs-klasse. Eine Testklasse wird erzeugt, indem auf einer potenziellen Bezugsklasse ein Rechtsklick ausgeführt wird und aus dem Kontextmenü der Eintrag TESTKLASSE ERZEUGEN selektiert wird. Beachten Sie, dass **Produkt** bereits eine Testklasse besitzt, sodass dieser zusätzliche Eintrag nicht im Kontextmenü der Klasse erscheint. Im Kontextmenü der Klasse **Kommentar** hingegen erscheint er, da dieser Klasse momentan keine Testklasse zugeordnet ist.

Der entscheidende Punkt an einer Testklasse ist, dass sie sowohl Quelltext enthält, der Tests auf einer Bezugsklasse ausführt, als auch Code, mit dem der Erfolg dieser Tests überprüft wird. Hier ist beispielsweise eine Anweisung aus **testInit**, die überprüft, dass der Preis des Produkts zu diesem Zeitpunkt 1000 ist:

```
assertEquals(1000, produkt1.gibPreis());
```

Wenn solche Tests ausgeführt werden, kann BlueJ die Ergebnisse in dem Fenster anzeigen, das in Abbildung 9.4 gezeigt wird.

Im nächsten Abschnitt werden wir diskutieren, wie BlueJ Sie bei der automatischen Erstellung von Regressionstest unterstützt, sodass Sie Ihre eigenen automatisierten Tests erstellen können.

Übung 9.13 Erzeugen Sie eine Testklasse für die Klasse **Kommentar** im Projekt *OnlineShop-JUnit*.

Übung 9.14 Welche Methoden werden automatisch erzeugt, wenn eine neue Testklasse angelegt wird?

9.4.3 Tests aufzeichnen

Wir haben bereits zu Beginn von Abschnitt 9.4 gesagt, dass Testautomatisierung wünschenswert ist, weil das manuelle Durchführen und Neu-Durchführen von Tests ein zeitaufwendiger Prozess ist. BlueJ ermöglicht, die Effektivität von interaktiv durchgeführten Tests mit der Stärke automatisierter Tests zu kombinieren, indem manuell durchgeführte Tests aufgezeichnet und später für Regressionstests wieder abgespielt werden können. Die Klasse **ProduktTest** aus dem Projekt *OnlineShop-JUnit* wurde auf diese Weise erstellt.

Nehmen Sie an, wir möchten die Methode **kommentarHinzufuegen** der Klasse **Produkt** gründlich testen. Diese Methode versucht, Kundenkommentare hinzuzufügen, wenn sie gültig sind. Wir möchten verschiedene Tests durchführen, beispielsweise wollen wir

- einen ersten Kommentar zu einer leeren Kommentarliste hinzufügen (positiv)
- weitere Kommentare zu bereits bestehenden Kommentaren hinzufügen (positiv)
- versuchen, einen Kommentar von einem Autor hinzuzufügen, der bereits einen Kommentar abgegeben hat (negativ)
- versuchen, einen Kommentar mit einer ungültigen Bewertung abzugeben (negativ).

Der erste Test existiert bereits in der Klasse **ProduktTest**. Im Folgenden werden wir beschreiben, wie wir den nächsten Test unter Verwendung des Projekts *OnlineShop-JUnit* erzeugen.

In BlueJ wird ein Test aufgezeichnet, indem eine Aufzeichnung gestartet wird, der Test manuell ausgeführt wird und dann das Ende der Aufzeichnung signalisiert wird. Der erste Schritt erfolgt über das Kontextmenü der Testklasse. Auf diese Weise weiß BlueJ, in welcher Klasse der neue Test gespeichert werden soll. Selektieren Sie **ERZUEGE TESTMETHODE...** aus dem Kontextmenü der Klasse **ProduktTest**. Sie werden daraufhin nach dem Namen für die Testmethode gefragt. Per Konvention beginnen wir den Namen mit dem Präfix **test**. Bei der Erzeugung einer Methode, die das Hinzufügen von zwei Kommentaren prüft, könnten wir beispielsweise als Methodenamen **testZweiKommentare** vergeben.¹

¹ Frühere Versionen von JUnit (bis Version 3) schrieben vor, dass die Methodennamen mit dem Präfix „test“ beginnen mussten. In aktuellen Versionen ist dies nicht mehr erforderlich.

Konzept

Eine **Zusicherung** ist ein boolescher Ausdruck für eine Bedingung, von der wir erwarten, dass sie wahr ist. Wenn sie nicht wahr ist, dann sagen wir, dass die Zusicherung fehlschlägt. Dies weist auf einen Fehler in unserem Programm hin.

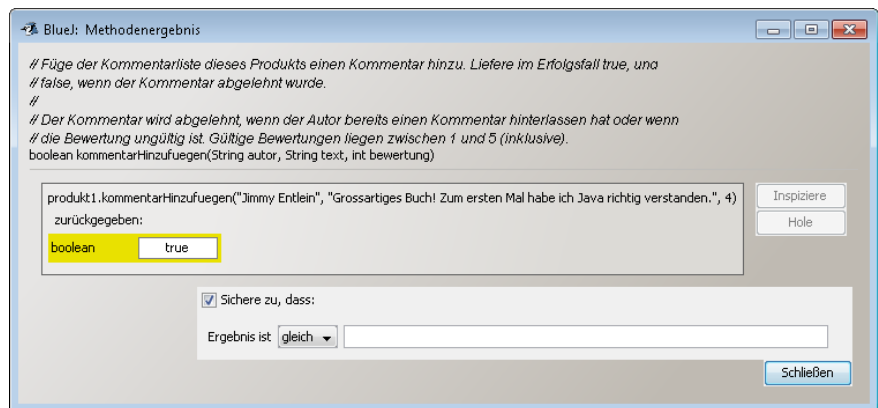
Abbildung 9.5
Der Dialog für Methodenergebnisse mit einer Unterstützung für Zusicherungen.

Sobald Sie einen Namen eingegeben und Ok geklickt haben, erscheint ein roter Kreis links vom Klassendiagramm und die Knöpfe **BEENDEN** und **ABBRECHEN** werden aktiviert. Mit **BEENDEN** können Sie eine Testerzeugung abschließen, mit **ABBRECHEN** können Sie sie verwerfen.

Sobald eine Aufzeichnung gestartet wurde, führen wir einfach die Aktionen durch, die wir in einem normalen manuellen Test vornehmen würden.

- Erzeugen eines **Produkt**-Objekts.
- Einen Kommentar zu dem Produkt hinzufügen.

Sobald **kommentarHinzufuegen** aufgerufen wurde, erscheint ein neues Dialogfenster (Abbildung 9.5). Dieses ist eine erweiterte Version des normalen Fensters für Methodenergebnisse und spielt eine wichtige Rolle bei der Erzeugung automatisierter Tests. In diesem Fenster können Sie angeben, welches Ergebnis der Methodenaufruf *haben sollte*. Eine in dieser Weise formalisierte Erwartung wird als *Zusicherung* (*assertion*) bezeichnet.



In diesem Fall erwarten wir, dass die Methode den Wert **true** zurückliefert, entsprechend wollen wir bei unserem Test eine Überprüfung durchführen, dass dies wirklich so ist. Wir selektieren also die Checkbox **SICHERE ZU, DASS**, geben **true** in den Dialog ein und klicken den **SCHLIEßEN**-Knopf.

- Fügen Sie zu Ihrem Produkt einen zweiten Kommentar hinzu. Achten Sie darauf, dass die Kommentare gültig sind (sie stammen jeweils von unterschiedlichen Autoren und ihre Bewertung ist gültig). Sichern Sie zu, dass das Ergebnis auch für den zweiten hinzugefügten Kommentar **true** ist.
- Wir gehen jetzt davon aus, dass zwei Kommentare existieren. Um zu testen, dass dies tatsächlich der Fall ist, rufen Sie die Methode **gibAnzahlKommentare** auf und sichern Sie zu, dass das Ergebnis 2 ist.

Da dies der letzte Schritt unseres Tests ist, klicken wir anschließend den **BEENDEN**-Knopf um die Aufzeichnung zu beenden. An dieser Stelle fügt BlueJ nun Quelltext für unsere neue Testmethode **testZweiKommentare** in die Klasse **ProduktTest** ein, übersetzt die Klasse und löscht die Objektleiste. Die resultierende generierte Methode ist in Listing 9.2 zu sehen.

Es ist zu erkennen, dass die Methode Anweisungen enthält, die die Aktionen reproduzieren, die während der Aufzeichnung durchgeführt wurden: Ein Objekt der Klasse

Produkt wird erzeugt und die Methoden **kommentarHinzufuegen** und **gibAnzahlKommentare** werden aufgerufen. Der Aufruf **assertEquals** überprüft, ob das von diesen Methoden zurückgelieferte Ergebnis zum erwarteten Wert passt. Darüber hinaus sehen Sie vor der Methode ein neues Konstrukt **@Test**. Damit wird gekennzeichnet, dass es sich bei dieser Methode um eine Testmethode handelt.

```
@Test
public void testZweiKommentare()
{
    Produkt produkt1 = new Produkt("Java-Buch", 12345);
    assertEquals(true, produkt1.kommentarHinzufuegen("Jimmy Entlein",
                                                    "Geniales Buch", 4));
    assertEquals(true, produkt1.kommentarHinzufuegen("Fred", "Mag es", 2));
    assertEquals(2, produkt1.gibAnzahlKommentare());
}
```

Listing 9.2

Eine automatisch generierte Testmethode.

In den folgenden Übungen können Sie diesen Prozess selbst ausprobieren. In einer Übung sehen Sie auch, was passiert, wenn der tatsächliche Wert nicht dem erwarteten entspricht.

Übung 9.15 Erzeugen Sie einen Test, der prüft, dass **kommentarHinzufuegen** den Wert **false** liefert, wenn ein Kommentar vom selben Autor bereits existiert.

Übung 9.16 Erstellen Sie den folgenden negativen Test für die Grenzfälle bei den Bewertungswerten. Das heißt, testen Sie die Werte 0 und 6 als Bewertung (die gerade außerhalb des gültigen Bereichs liegen). Wir erwarten, dass diese **false** liefern, sodass Sie im Dialogfenster **METHODENERGEBNIS false** zusichern müssen. Sie werden feststellen, dass einer davon tatsächlich (fälschlicherweise) **true** liefert. Dies ist einer der Fehler, den wir zuvor durch interaktives Testen festgestellt haben. Achten Sie darauf, dass Sie trotzdem **false** zusichern. Die Zusage gibt das *erwartete* Ergebnis an und nicht das *tatsächliche* Ergebnis.

Übung 9.17 Führen Sie alle Tests erneut durch. Im Dialogfenster **TESTERGEBNIS** können Sie sich davon überzeugen, dass der fehlgeschlagene Test angezeigt wird. Selektieren Sie den fehlgeschlagenen Test in der Liste. Welche Optionen stehen Ihnen zur Verfügung, um die Details des fehlgeschlagenen Tests zu untersuchen?

Übung 9.18 Erzeugen Sie eine Testklasse, die **Kommentar** als ihre Bezugsklasse hat. Erzeugen Sie einen Test, der prüft, ob der Autor und die Bewertungsdetails nach der Erzeugung korrekt gespeichert wurden. Zeichnen Sie getrennte Testmethoden darin auf, die überprüfen, dass die Methode **ablehnen** und **zustimmen** wie erwartet funktionieren.

Übung 9.19 Erstellen Sie Tests für die Klasse **Produkt**, die testet, ob die Methode **findeHilfreichstenKommentar** wie erwartet arbeitet. Beachten Sie, dass diese Methode ein **Kommentar**-Objekt liefert. Während des Testens können Sie den Knopf **HOLE** im Dialogfenster **Methodenergebnis** dazu nutzen, um das Ergebnisobjekt in die Objektleiste zu „holen“, wo Sie dann weitere Methodenaufrufe machen und Zusicherungen für dieses Objekt hinzufügen können. Das erlaubt es, dass zurückgelieferte Kommentarobjekt zu identifizieren (z.B. durch Prüfen seines Autors). Sie können auch zusichern, dass das Ergebnis *null* beziehungsweise *nicht null* ist (je nachdem, was Sie erwarten).

9.4.4 Testgerüste

Konzept

Ein **Testgerüst** besteht aus einer Menge von Objekten in einem definierten Zustand, die als Grundlage für einen Modultest dienen.

Bei der Erstellung von Testmethoden werden Sie häufig beobachten, dass Sie immer wieder ähnliche Objekte für jede Testmethode erstellen. Beispielsweise benötigen Sie für jeden Test der Klasse **Produkt** mindestens ein Objekt der Klasse **Produkt**, das Sie initialisieren müssen, oft indem Sie ein oder mehrere Kommentare hinzufügen. Ein Objekt oder eine Gruppe von Objekten, die in mehr als einem Test verwendet werden, werden als *Testgerüst* bezeichnet. Zwei Menüeinträge einer Testklasse ermöglichen uns, in BlueJ mit Testgerüsten zu arbeiten: **OBJEKTZUSTAND SPEICHERN** und **OBJEKTZUSTAND WIEDERHERSTELLEN**. Erzeugen Sie in Ihrem Projekt zwei **Produkt**-Objekte auf der Objektleiste. Eines davon soll keine Kommentare und das andere zwei Kommentare erhalten. Selektieren Sie nun den Eintrag **OBJEKTZUSTAND SPEICHERN** im Kontextmenü von **ProduktTest**. Die Objekte verschwinden von der Objektleiste und im Quelltext der Klasse **ProduktTest** können Sie erkennen, dass die Methode **setUp** ungefähr so aussieht wie in Listing 9.3, in dem **produkt1** und **produkt2** als Datenfelder der Testklasse definiert sind.

Listing 9.3

Der Aufbau eines Testgerüsts.

```
/**
 * Setzt das Testgerüst für den Test.
 *
 * Wird vor jeder Testfallmethode aufgerufen.
 */
@Before
public void setUp()
{
    produkt1 = new Produkt("Java-Buch", 12345);
    produkt2 = new Produkt("Harry Potter", 1250);
    produkt2.kommentarHinzufuegen("Fred", "Weltbestes Buch", 5);
}
```

Die Methode **setUp** wird automatisch jedes Mal aufgerufen, bevor eine Testmethode aufgerufen wird. (Die Kennzeichnung **@Before** über dem Methodenkopf stellt dies sicher.) Dies bedeutet, dass eine individuelle Testmethode nun nicht mehr ihr eigenes Testgerüst aufzubauen braucht.

Sobald wir ein Testgerüst für eine Testklasse definiert haben, vereinfachen sich auch die Aufzeichnungen weiterer Tests: Wenn wir eine neue Testmethode erzeugen, werden die Objekte des Testgerüsts automatisch erzeugt und auf die Objektleiste gelegt – es besteht keine Notwendigkeit mehr, jedes Mal neue Testobjekte interaktiv zu erzeugen.

Wenn wir ein Testgerüst um weitere Objekte ergänzen wollen, brauchen wir lediglich **OBJEKTZUSTAND WIEDERHERSTELLEN** zu selektieren, die zusätzlichen Objekte erzeugen und dann **OBJEKTZUSTAND SPEICHERN** auszuführen. Natürlich können wir auch einfach die Methode **setUp** editieren und weitere Datenfelder direkt in der Testklasse einfügen.

Testautomatisierung ist ein mächtiges Konzept, denn es macht es wahrscheinlicher, dass überhaupt Tests geschrieben werden, und es macht wahrscheinlicher, dass diese im Laufe der Programmentwicklung ausgeführt und wiederholt werden. Sie sollten sich angewöhnen, Testklassen so früh wie möglich bei der Arbeit an einem Projekt zu schreiben und diese auch ständig aktuell zu halten. In Kapitel 14 werden wir das Konzept der Zusicherungen im Zusammenhang mit Fehlerbehandlung erneut aufgreifen.

Übung 9.20 Fügen Sie in Ihrem Projekt so lange weitere automatisierte Tests ein, bis Sie ausreichend Vertrauen in die korrekte Arbeitsweise der zu testenden Klassen haben. Testen Sie sowohl positiv als auch negativ. Wenn Sie Fehler entdecken sollten, dann erstellen Sie Tests, die Sie vor dem erneuten Auftreten dieser Fehler in einer späteren Version schützen.

Im Abschnitt 9.6 beschäftigen wir uns mit Debugging – die Aktivität, die einsetzt, wenn wir feststellen, dass es einen Fehler gibt, den wir finden und beheben müssen.

9.5 Refactoring bei Streams einsetzen (fortgeschritten)

In den vorherigen Abschnitten dieses Buchs über fortgeschrittene Konzepte haben wir Streams als Alternative zu Schleifen bei der Verarbeitung von Sammlungen eingeführt. Das *OnlineShop*-Projekt bietet die Gelegenheit, das Umformen traditioneller Schleifen durch den Einsatz von Streams weiter zu üben. Wenn Sie die vorigen Übungen zu den fortgeschrittenen Themen durchgearbeitet haben, wird Ihnen vielleicht auch die folgende Spaß machen.

Übung 9.21 Schreiben Sie alle Schleifen in der Klasse **Produkt** unter Verwendung von Streams neu. Wenn Sie fertig sind, führen Sie Ihre Tests erneut aus, um sich davon zu überzeugen, dass auch nach Ihren Veränderungen die Klasse noch wie beabsichtigt funktioniert.

9.6 Debugging

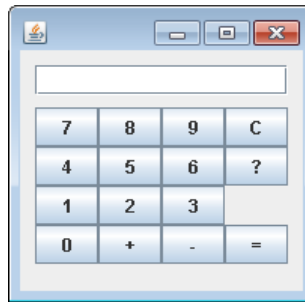
Testen ist wichtig und gutes Testen trägt dazu bei, Fehler festzustellen. Doch Testen allein reicht nicht aus. Nachdem wir einen Fehler festgestellt haben, müssen wir seine Ursache finden und diese beseitigen. Hier kommt das Debugging ins Spiel.

Für unsere Diskussion der verschiedenen Ansätze zum Debuggen bedienen wir uns eines hypothetischen Szenarios. Angenommen, Sie wurden gebeten, in einem Projektteam mitzuarbeiten, das die Implementierung eines Softwarerechners vornimmt (Abbildung 9.6). Sie werden ausgewählt, weil ein Kernmitglied des Programmierteams – Hacker T. Largebrain – gerade auf eine Managementposition in einem anderen Projekt versetzt wurde. Bevor er Ihr neues Team verlassen hat, hat er beteuert, dass seine Implementierung von dem Teil des Rechners, für den er verantwortlich ist, abgeschlossen ist und vollständig getestet wurde. Er hat sogar etwas Testsoftware geschrieben, mit der dies belegt werden soll. Sie wurden gebeten, diese Klasse zu übernehmen und sicherzustellen, dass sie ausreichend kommentiert ist, bevor sie mit den Klassen der anderen Teammitglieder integriert wird.

Die Software für den Rechner wurde sorgfältig entworfen, um die Benutzungsschnittstellen von der Rechnerlogik zu trennen, sodass der Rechner später auch in anderen Kontexten verwendet werden kann. Die erste Version, die wir hier betrachten, wird mit einer grafischen Benutzungsschnittstelle wie in Abbildung 9.6 ausgeführt. In späteren Erweiterungen zu dem Projekt wollen wir jedoch erreichen, dass eben diese Rechnerimplementierung in einem Webbrowser oder auf einem mobilen Gerät ausgeführt werden kann. Vorbereitend haben wir deshalb die Anwendung in mehrere Klassen geteilt, vor allem **GrafischeSchnittstelle**, die die grafische Benutzungsschnittstelle implementiert, und **Recheneinheit**, die die Rechnerlogik implementiert. Für eben diese letzte Klasse war Hacker verantwortlich. Diese Klasse sollte unverändert bleiben, wenn der Rechner mit einer anderen Benutzungsschnittstelle ausgeführt wird.

Abbildung 9.6

Die Benutzungsschnittstelle eines Softwarerechners.



Um festzustellen, wie die Klasse **Recheneinheit** von anderen Klassen verwendet wird, ist es hilfreich, einen Blick auf ihre *Schnittstelle* zu werfen – wobei hier nicht die *Benutzungsschnittstelle*, sondern die *Klassenschnittstelle* gemeint ist. Diese doppelte Bedeutung des Begriffs *Schnittstelle* ist bedauernswert, aber es ist wichtig, beide Bedeutungen zu verstehen.

Die Klassenschnittstelle ist die Zusammenfassung der öffentlichen Methodenköpfe, die andere Klassen benutzen können. Genau das können andere Klassen sehen und darüber können sie mit unserer Klasse interagieren. Die Schnittstelle wird in der **javadoc**-Dokumentation einer Klasse sowie in der Ansicht *Dokumentation* im Editor angezeigt. Listing 9.4 zeigt die Schnittstelle der Klasse **Recheneinheit**.

Listing 9.4

Die Schnittstelle einer arithmetischen Einheit.

```
/**
 * @return der Wert, der aktuell in der Anzeige gezeigt wird
 */
public int gibAnzeigewert();

/**
 * Eine Zifferntaste wurde getippt.
 * @param ziffer die einzelne Ziffer.
 */
public void zifferGetippt(int ziffer);

/**
 * Die '+'-Taste wurde getippt.
 */
public void plus();

/**
 * Die '-'-Taste wurde getippt.
 */
public void minus();
```

```
/**
 * Die '='-Taste wurde getippt.
 */
public void gleich();
```

```
/**
 * Die C-Taste (für 'Clear') wurde getippt.
 */
public void clear();
```

Solch eine Schnittstelle kann geschrieben werden, bevor die vollständigen Klassen implementiert sind. Sie bildet so etwas wie einen simplen Vertrag zwischen der Klasse **Recheneinheit** und den anderen Teilen des Programms, die diese Klasse benutzen wollen. Die Klasse **Recheneinheit** bietet die Implementierung dieser Schnittstelle. Die Schnittstelle beschreibt den minimalen Satz an Methoden, die in der logischen Komponente implementiert werden, und für jede Methode werden der Ergebnistyp und die Parameter definiert. Beachten Sie, dass die Schnittstelle nichts darüber aussagt, was in der implementierenden Klasse intern abläuft, wenn beispielsweise die Plus-Taste gedrückt wurde; das bleibt den Programmierern überlassen, die dies implementieren. Zusätzlich kann die implementierende Klasse weitere Methoden enthalten, die hier nicht aufgeführt sind.

In den folgenden Abschnitten werden wir uns Hackers Implementierungsversuch für diese Schnittstelle ansehen. In diesem Fall entscheiden wir uns, bevor wir mit dem Dokumentieren beginnen, dass wir den besten Einblick in Hackers Software bekommen, wenn wir den Quelltext untersuchen und das Verhalten der Objekte analysieren.

9.7 Kommentierung und Programmierstil

Öffnen Sie das Projekt *Recheneinheit*, um die Klassen zu untersuchen. Es handelt sich dabei um Hackers Version des Projekts, das nur die **Recheneinheit** und eine Testklasse enthält. Die Klasse **RecheneinheitTester** übernimmt an diesem Punkt der Entwicklung die Rolle der Benutzungsschnittstelle. Dies demonstriert einen weiteren Vorteil von wohldefinierten Schnittstellen zwischen Klassen: Es wird einfacher, Stellvertreterklassen für andere Klassen, sogenannte *Mock-Ups*, für Testzwecke zu definieren.

Wenn Sie sich die Klasse **Recheneinheit** ansehen, dann werden Sie feststellen, dass ihr Autor Wert auf einige wichtige Aspekte von gutem Programmierstil gelegt hat:

- Die Klasse hat einen mehrzeiligen Klassenkommentar bekommen, der den Zweck der Klasse beschreibt. Auch Autor und Versionsnummer sind angegeben.
- Jede Methode der Schnittstelle hat einen Kommentar, der den Zweck der Methode, ihre Parameter und ihren Ergebnistyp beschreibt. Dies macht es sicherlich leichter, eine Projektdokumentation für die Schnittstelle so zu generieren, wie in Kapitel 6 besprochen.
- Das Layout der Klasse ist einheitlich, mit angemessenen Einrückungen für die geschachtelten Blöcke und Kontrollstrukturen.
- Es wurden sinnvolle Namen für Variablen und Methoden gewählt.

Auch wenn das Einhalten dieser Konventionen während der Implementierung etwas Zeit kostet, können sie doch von enormem Nutzen sein, wenn jemand anderes diesen Quelltext verstehen muss (so, wie wir in diesem Szenario) oder wenn der Autor selbst sich nach einer längeren Pause wieder an den Zweck der Klasse erinnern muss.

Dabei fällt ein weiteres Detail ins Auge, das verdächtig erscheint: Hacker hat keine spezielle Modultest-Klasse verwendet, um seine Tests durchzuführen, sondern seine eigene Testklasse geschrieben. Da wir wissen, dass BlueJ Modultests unterstützt, fragen wir uns natürlich warum.

Eigene Testklassen zu schreiben muss kein Nachteil sein und selbst geschriebene Testklassen müssen nicht notwendigerweise schlechter sein. Trotzdem sind wir jetzt etwas misstrauisch geworden. Wusste Hacker wirklich, was er tat? Wir werden bald auf diese Frage zurückkommen.

Nun, möglicherweise ist Hacker wirklich so gut, wie er von sich selbst glaubt, und Sie haben nicht viel zu tun, um diese Klasse mit den übrigen zu integrieren! Führen Sie die folgenden Übungen durch, um dies zu überprüfen.

Übung 9.22 Stellen Sie sicher, dass alle Klassen übersetzt sind, und erzeugen Sie dann in BlueJ ein Objekt der Klasse **RecheneinheitTester**. Rufen Sie die Methode **allesTesten** auf. Was wird auf der Konsole ausgegeben? Glauben Sie der Aussage in der letzten Zeile?

Übung 9.23 An dem Objekt der letzten Übung sollen Sie nun die Methode **testPlus** aufrufen. Welches Ergebnis liefert sie? Ist dies das gleiche Ergebnis wie bei **allesTesten**? Rufen Sie **testPlus** noch einmal auf. Welches Ergebnis wird nun geliefert? Sollte die Methode immer die gleiche Antwort liefern? Wenn ja, wie sollte diese Antwort lauten? Überprüfen Sie dazu im Quelltext die Umsetzung der Methode **testPlus**.

Übung 9.24 Wiederholen Sie die vorige Übung mit der Methode **testMinus**. Liefert diese immer das gleiche Ergebnis?

Durch diese Experimente sollten bei Ihnen nun die Alarmglocken läuten und Sie warnen, dass mit der Klasse **Recheneinheit** etwas nicht stimmt. Es sieht so aus, als ob die Klasse Fehler enthält. Aber welche Fehler sind das und wo sind sie versteckt? In den folgenden Abschnitten werden wir einige Wege aufzeigen, wie wir Fehler in einer Klasse lokalisieren können.

9.8 Manuelle Ausführung

Konzept

Eine **manuelle Ausführung** ist das zeilenweise Durchgehen durch einen Quelltextabschnitt, bei dem Zustandsänderungen und das Verhalten einer Anwendung beobachtet werden.

Manuelle Ausführung (*walkthrough*) ist eine Technik, die eindeutig zu wenig eingesetzt wird. Das liegt möglicherweise daran, dass sie eine vergleichsweise „untechnische“ Technik zum Testen und zur Fehlerbeseitigung ist. Sie ist allerdings trotzdem – oder gerade deshalb – sehr nützlich. Eine manuelle Ausführung erfordert, dass Sie den Quelltext der zu untersuchenden Klasse ausdrucken und sich dann weit weg von Ihrem Computer mit dem Ausdruck hinsetzen! Es ist sehr leicht, sehr viel Zeit vor einem Computerbildschirm sitzend zu verbringen und nur wenig Fortschritt bei einem Programmierproblem zu machen. Ein Ortswechsel und ein Wechsel der Betrachtung können bei einer Blockade oft helfen, das Problem aus einer völlig anderen Sicht anzugehen. Wir haben schon sehr oft erlebt, dass uns auf dem Weg zum Mittagessen oder auf der Fahrt nach Hause die Lösung eingefallen ist, nach der wir in ermüdenden Stunden an der Tastatur gesucht haben.

Eine manuelle Ausführung schließt sowohl das Lesen von Klassentexten als auch das Verfolgen des Kontrollflusses zwischen Klassen und Objekten ein. Dies hilft uns, die Interaktion der Objekte und ihre interne Arbeitsweise zu verstehen. Letztendlich ist eine manuelle Ausführung eine Papier-und-Bleistift-Simulation eines Programmablaufs auf einem Computer. In der Praxis sollte dabei nur ein kleiner Teil einer Anwendung betrachtet werden, etwa eine logische Einheit von Aktionen oder sogar nur ein einzelner Methodenaufruf.

9.8.1 Eine Ausführung auf erster Ebene

Wir werden die Technik der manuellen Ausführung anhand des Projekts *Recheneinheit* demonstrieren. Sie können sich die Klassendefinitionen von **Recheneinheit** und **RecheneinheitTester** ausdrucken, um die folgende Diskussion nachzuvollziehen.

Wir beginnen mit einer Untersuchung der Methode **testPlus** der Klasse **RecheneinheitTester**, da diese eine logische Einheit von Aktionen darstellt. Sie sollte uns erleichtern zu verstehen, wie die einzelnen Methoden der Klasse **Recheneinheit** zusammenspielen, um der Rolle der Recheneinheit eines Taschenrechners gerecht zu werden. Während wir uns durch den Quelltext arbeiten, werden wir die Fragen notieren, die uns dabei in den Sinn kommen.

1 Für diese erste Stufe wollen wir nicht zu sehr ins Detail gehen. Wir wollen einfach nur sehen, wie die Methode **testPlus** ein **Recheneinheit**-Objekt verwendet, ohne die Interna des Objekts zu betrachten. Durch unsere vorherigen Experimente wissen wir, dass es Fehler gibt, aber wir wissen nicht, ob diese im Tester oder der Recheneinheit liegen. Im ersten Schritt wollen wir deshalb untersuchen, ob der Tester die Recheneinheit korrekt benutzt.

2 Wir stellen fest, dass die erste Anweisung in **testPlus** davon ausgeht, dass das Datenfeld **recheneinheit** bereits eine gültige Referenz auf ein Objekt enthält:

```
recheneinheit.clear();
```

Wir können diese Annahme bestätigen, indem wir uns den Konstruktor der Testklasse ansehen. Ein häufig gemachter Fehler bei Datenfeldern ist, dass sie nicht korrekt initialisiert werden, weder bei ihrer Deklaration noch in einem Konstruktor. Wenn wir ein Datenfeld benutzen wollen, das keine Referenz auf ein Objekt enthält, erhalten wir üblicherweise eine **NullPointerException**.

3 Der Aufruf von **clear** als erste Anweisung scheint der Versuch zu sein, die Recheneinheit in einen gültigen Anfangszustand zu versetzen, damit sie Instruktionen für das Durchführen von Rechenoperationen entgegennehmen kann. Das scheint vernünftig zu sein – vergleichbar mit dem Drücken einer Reset- oder Clear-Taste an einem echten Taschenrechner. An diesem Punkt betrachten wir die Rechner-Klasse nicht näher, um die exakte Arbeitsweise der Methode **clear** herauszufinden. Das kann warten, bis wir genügend Vertrauen in die Aktionen des Testers haben. Stattdessen machen wir uns nur eine kurze Notiz, dass wir noch prüfen müssen, ob die Methode **clear** den Rechner wie erwartet in einen korrekten Anfangszustand versetzt.

4 Die nächste Anweisung in **testPlus** ist die Eingabe einer Ziffer über die Methode **zifferGetippt**:

```
rechner.zifferGetippt(3);
```

Auch dies scheint vernünftig, da der erste Schritt einer Berechnung die Eingabe des ersten Operanden ist. Auch hier: Wir überprüfen nicht, was die Recheneinheit mit der Ziffer macht. Wir nehmen einfach an, dass sie diese irgendwo speichert, um sie für die spätere Berechnung zu verwenden.

- 5** Die nächste Anweisung ruft **plus** auf. Somit wissen wir, dass der volle Wert des ersten Operanden 3 ist. Wir können dies kurz auf dem Ausdruck notieren oder wir können einen Haken an die Zusicherung in einem der Kommentare von **testPlus** machen. Außerdem sollten wir notieren, dass die auszuführende Operation die Addition ist. Dies scheint trivial zu sein, aber es kommt sehr leicht vor, dass der Kommentar in einer Klasse nicht mit dem kommentierten Quelltext übereinstimmt. Das gleichzeitige Überprüfen der Kommentare kann uns somit davor schützen, später von ihnen in die Irre geleitet zu werden.
- 6** Als Nächstes wird eine weitere einzelne Ziffer als der rechte Operand eingegeben, wieder durch einen Aufruf von **zifferGetippt**.
- 7** Die Ausführung der Addition wird durch den Aufruf von **gleich** angestoßen. Wir notieren hier kurz, dass diese Methode, so, wie sie in **testPlus** benutzt wird, scheinbar nicht das erwartete Ergebnis der Berechnung zurückliefert. Das ist ein weiterer Punkt, den wir bei der näheren Betrachtung von **Recheneinheit** prüfen sollten.
- 8** Die letzte Anweisung in **testPlus** fordert den Wert an, der in der Anzeige des Taschenrechners erscheinen sollte:

```
return recheneinheit.gibAnzeigewert();
```

Anscheinend soll dies das Ergebnis der Berechnung sein, aber wir können es nicht wissen, bevor wir uns nicht ausführlich die Klasse **Recheneinheit** angesehen haben. Auch hier machen wir eine kurze Notiz, dass wir das überprüfen sollten.

Nachdem wir uns die Methode **testPlus** angesehen haben, haben wir ausreichend Vertrauen in ihre Implementierung gewonnen: Sie benutzt die Recheneinheit korrekt, indem sie eine Reihe von Tastendrücken simuliert, die eine komplette Berechnung durchführen. Wir könnten anmerken, dass diese Methode nicht sonderlich ehrgeizig implementiert ist – beide Operanden bestehen aus nur einer einzelnen Ziffer und es wird nur ein einzelner Operator verwendet. Dies ist jedoch nicht ungewöhnlich für Testmethoden, denn die Basisfunktionalität sollte auf jeden Fall getestet werden, bevor komplexere Kombinationen untersucht werden. Trotzdem sollten noch einige komplexere Tests hinzugefügt werden.

Übung 9.25 Führen Sie selbst eine ähnliche manuelle Ausführung mit der Methode **testMinus** durch. Kommen Sie dadurch auf weitere Fragen, die Sie bei der Untersuchung der Klasse **Recheneinheit** klären wollen?

Bevor wir uns die Klasse **Recheneinheit** ansehen, sollten wir kurz einen Blick auf die Methode **allesTesten** werfen, um zu sehen, wie diese die von uns bereits untersuchten Methoden **testPlus** und **testMinus** benutzt. Dabei können wir Folgendes beobachten:

- 1 Die Methode **allesTesten** besteht aus einer einfachen Sequenz von **print**-Anweisungen.
- 2 Sie enthält jeweils einen Aufruf der Methoden **testPlus** und **testMinus**, deren Rückgabewerte zur Betrachtung ausgegeben werden. Wir stellen fest, dass es keinen Hinweis für den Benutzer gibt, welche Werte korrekt wären. Dies macht es dem Benutzer schwer zu entscheiden, ob die Ergebnisse korrekt sind.
- 3 Wir stellen fest, dass die letzte Anweisung kühn verkündet:
Alle Tests erfolgreich.

Aber die Methode enthält keine Prüfung, die die Korrektheit dieser Aussage absichert! Es sollten aber unbedingt zum einen die erwarteten Werte angezeigt werden und zum anderen sollte überprüft werden, ob die berechneten Werte mit diesen übereinstimmen. Das ist etwas, das wir unbedingt korrigieren sollten, wenn wir zu dieser Klasse zurückkehren.

An dieser Stelle sollten wir uns durch den letzten Punkt nicht dazu verleiten lassen, Änderungen vorzunehmen, die nicht mit dem eigentlichen Problem zusammenhängen. Wenn wir solche Änderungen machen würden, könnten wir das eigentliche Problem möglicherweise verdecken. Eine der zentralen Voraussetzungen für erfolgreiche Fehlerbeseitigung ist, den gesuchten Fehler leicht und wiederholbar auftreten lassen zu können. Sobald das erreicht ist, fällt die Beurteilung einer möglichen Korrektur viel leichter.

Nachdem wir die Testklasse untersucht haben, können wir uns nun dem Quelltext der Klasse **Recheneinheit** zuwenden. Wir sind ausgerüstet mit einer Reihe zu untersuchender Methodenaufrufe aus der manuellen Ausführung der Methode **testPlus** und mit einigen Fragen, die dabei aufgetreten sind.

9.8.2 Den Zustand kontrollieren

Ein Objekt der Klasse **Recheneinheit** unterscheidet sich in seiner Art recht deutlich von seinem Tester, denn es ist ein vollständig passives Objekt. Es stößt von sich aus keine Aktionen an, sondern reagiert lediglich auf externe Aufrufe von Methoden. So etwas ist ein typisches Server-Verhalten (englisch für „Dienstleister“). Server-Objekte machen häufig primär von ihrem internen Zustand abhängig, wie sie auf Methodenaufrufe reagieren. Dies gilt insbesondere für unsere **Recheneinheit**. Deshalb ist ein wichtiger Bestandteil einer manuellen Ausführung die Kontrolle ihres internen Zustands. Eine Möglichkeit dazu ist das Notieren der Datenfelder und ihrer Werte in einer handgeschriebenen Tabelle (Tabelle 9.1). Neue Zeilen können hinzugefügt werden, sodass ein Protokoll der Werte nach jedem Methodenaufruf entsteht.

Tabelle 9.1
Informelle Tabel-
lierung der Zustände
eines Objekts.

Aufgerufene Methode	anzeigewert	linker-Operand	rechter-Operand
Anfangszustand	0	0	' '
clear	0	0	' '
zifferGetippt(3)	3	0	' '

Diese Technik unterstützt sehr leicht die Überprüfung, ob etwas schiefgeht. Sie ermöglicht auch, die Zustände nach zwei Aufrufen derselben Methode zu überprüfen.

- 1

Zu Beginn unserer Ausführung der Recheneinheit halten wir ihren Anfangszustand fest, wie in der ersten Zeile in Tabelle 9.1. Alle Datenfelder der Einheit werden im Konstruktor initialisiert. Wir haben bei der Ausführung des Testers festgestellt, dass die Objektinitialisierung wichtig ist, und halten an dieser Stelle fest, dass wir die Initialisierung noch überprüfen sollten – insbesondere, weil der Anfangswert von **letzterOperator** keinen gültigen Operator darzustellen scheint. Außerdem fragen wir uns, ob es einen *letzten* Operator geben sollte, bevor wirklich ein Operator in einer Berechnung auftritt. Durch das Aufschreiben dieser Fragen müssen wir sie nicht unmittelbar beantworten, sondern können sie wieder aufgreifen, wenn wir mehr über die Klasse wissen.
- 2

Im nächsten Schritt prüfen wir, ob ein Aufruf von **clear** den Zustand der Recheneinheit ändert. Die zweite Zeile in Tabelle 9.1 zeigt an, dass der Zustand an dieser Stelle unverändert bleibt, weil **anzeigewert** bereits den Wert null hatte. Aber wir notieren eine weitere Frage: Warum wird nur eines der Datenfelder in dieser Methode gesetzt? Wenn diese Methode eine Art „Reset“ implementieren soll, warum werden dann nicht alle Datenfelder zurückgesetzt?
- 3

Als Nächstes wird der Aufruf von **zifferGetippt** mit dem aktuellen Parameter 3 untersucht. Diese Methode multipliziert einen bereits bestehenden Wert von **anzeigewert** mit 10 und addiert dann die neue Ziffer. Dies modelliert korrekt das Anfügen einer weiteren Ziffer an das rechte Ende einer bereits existierenden Zahl. Die Methode verlässt sich darauf, dass **anzeigewert** einen vernünftigen Anfangswert von null hat, wenn die erste Ziffer eingetippt wird, und unsere Untersuchung von **clear** liefert uns die Gewissheit, dass dies der Fall ist. Diese Methode scheint in Ordnung zu sein.
- 4

Den Aufrufen in der Methode **testPlus** folgend, betrachten wir nun die Methode **plus**. Die erste Anweisung ist ein Aufruf der Methode **letztenOperatorAnwenden**. An dieser Stelle müssen wir uns entscheiden, ob wir weiterhin geschachtelte Aufrufe übergehen wollen oder ob wir das Vorgehen ändern und gleich ihre Implementierung betrachten. Ein kurzer Blick auf die Methode **letztenOperatorAnwenden** zeigt uns, dass sie recht kurz ist. Außerdem scheint sie offensichtlich den Zustand der Recheneinheit zu verändern, sodass wir den Zustand nicht weiter verfolgen könnten, wenn wir diesem Aufruf nicht nachgingen. Gehen wir ihm also nach. Wir sollten im Auge behalten, von wo wir

gekommen sind, also markieren wir die Stelle in **plus**, an der wir der **letztenOperatorAnwenden**-Methode folgen. Wenn das Verfolgen eines geschachtelten Aufrufs zu weiteren geschachtelten Aufrufen führt, brauchen wir mehr als eine einfache Markierung, um unseren Weg zurückzufinden. In einem solchen Fall ist es besser, die Aufrufpunkte mit aufsteigenden Nummern zu versehen – und diese Nummern nach einem Rücksprung wieder zu verwenden.

- 5** Die Methode **letztenOperatorAnwenden** gibt uns Aufschluss darüber, wie das Datenfeld **letzterOperator** verwendet wird. Sie scheint auch eine unserer früheren Fragen zu beantworten: ob ein Leerzeichen als Anfangswert für den letzten Operator ausreichend ist. Die Methode überprüft explizit, ob **letzterOperator** ein **+** oder ein **-** enthält, bevor sie ihn anwendet. Ein anderer Wert führt somit nicht zur Anwendung einer falschen Operation. Am Ende dieser Methode wird sich der Wert von **linkerOperand** verändert haben, also halten wir seinen neuen Wert in der Zustandstabelle fest.
- 6** Wieder zurück in der Methode **plus**, erhalten die beiden anderen Datenfelder ihre Werte, sodass die nächste Zeile in unserer Zustandstabelle (Tabelle 9.1) folgende Werte enthält:

```
plus03 '+'
```

Die Ausführung der Recheneinheit kann in ähnlicher Weise fortgesetzt werden: Zustandsänderungen notieren, Verhalten verstehen, neue Fragen stellen. Die folgenden Übungen sollten Ihnen beim Abschluss der manuellen Ausführung helfen.

Übung 9.26 Vervollständigen Sie die Zustandstabelle auf Basis der folgenden Aufrufsequenz, die der Methode **testPlus** entnommen ist:

```
zifferGetippt(4);
gleich();
gibAnzeigewert();
```

Übung 9.27 Bei der Ausführung der Methode **gleich**: Sind Sie hier ähnlich beruhigt über den Anfangswert von **letzterOperator**, wie wir es in der Methode **letztenOperatorAnwenden** waren?

Übung 9.28 Führen Sie manuell einen Aufruf der Methode **clear** unmittelbar nach dem Aufruf von **gibAnzeigewert** am Ende Ihrer Zustandstabelle aus und notieren Sie den neuen Zustand. Befindet sich die Recheneinheit im gleichen Zustand wie nach dem letzten Aufruf von **clear**? Wenn nicht: Welchen Einfluss könnte das Ihrer Meinung nach auf nachfolgende Berechnungen haben?

Übung 9.29 Nach den Erkenntnissen Ihrer Ausführung: Welche Änderungen sollten Ihrer Meinung nach an der Klasse **Recheneinheit** vorgenommen werden? Führen Sie diese Änderungen an Ihrer Papierversion durch und machen Sie dann eine weitere Ausführung. Sie brauchen nicht die Klasse **RecheneinheitTester** auszuführen, wiederholen Sie einfach die Aktionen ihrer Methode **allesTesten**.

Übung 9.30 Versuchen Sie eine Ausführung der folgenden Aufrufsequenz an Ihrer korrigierten Version der Recheneinheit:

```
clear();
zifferGetippt(9);
plus();
zifferGetippt(1);
minus();
zifferGetippt(4);
gleich();
```

Welches Ergebnis sollte das ergeben? Verhält sich die Recheneinheit korrekt und hinterlässt den korrekten Wert in **anzeigewert**?

9.8.3 Mündliche Ausführung

Eine weitere Anwendung von Ausführungen zur Fehlerbeseitigung kann darin bestehen, einer anderen Person die Arbeitsweise einer Klasse oder einer Methode zu erklären. Das kann zu ganz unterschiedlichen Ergebnissen führen:

- Die Person, der Sie den Quelltext erläutern, entdeckt den Fehler.
- Allein schon der Versuch, das gewünschte Verhalten eines Programmstücks in Worte zu fassen, macht Ihnen selbst klar, warum es sich nicht so verhält.

Dieser zweite Effekt ist so häufig zu beobachten, dass es oft nützlich sein kann, einer Person, die den Quelltext überhaupt nicht kennt, seine Arbeitsweise zu erklären – nicht mit der Erwartung, dass *diese Person* den Fehler findet, sondern *Sie selbst*!

9.9 Ausgabeanweisungen

Die vermutlich am häufigsten eingesetzte Technik zum Verstehen und zur Fehlerbeseitigung in Programmen – selbst unter erfahrenen Programmierern – ist das temporäre Einfügen von Ausgabeanweisungen. Ausgabeanweisungen sind sehr beliebt, weil es sie in den meisten Programmiersprachen gibt, weil sie jedem zur Verfügung stehen und weil sie sehr leicht mit jedem Editor eingefügt werden können. Es werden keine weiteren Sprachkonstrukte oder zusätzliche Software benötigt, um sie einzusetzen. Während der Ausführung eines Programms liefern diese zusätzlichen Ausgabeanweisungen dem Benutzer Informationen wie:

- welche Methoden aufgerufen wurden
- die Werte von Parametern
- die Reihenfolge, in der Methoden aufgerufen wurden
- die Werte von Datenfeldern und lokalen Variablen an strategisch wichtigen Stellen

Listing 9.5 zeigt ein Beispiel, wie die Methode **zifferGetippt** mit zusätzlichen Ausgabeanweisungen aussehen könnte. Solche Informationen sind insbesondere dann nützlich, wenn man sich ein Bild davon verschaffen will, wie der Objekt-

zustand durch verändernde Methoden beeinflusst wird. Um dies noch besser zu unterstützen, kann eine Methode speziell zur Fehlersuche eingefügt werden, die die aktuellen Werte aller Datenfelder ausgibt Listing 9.6 zeigt eine solche Methode `zustandAusgeben` für die Klasse `Recheneinheit`.

```
/**
 * Eine Zifferntaste wurde getippt.
 * @param ziffer die einzelne Ziffer
 */
public void zifferGetippt(int ziffer)
{
    System.out.println("zifferGetippt aufgerufen mit: "
        + ziffer);
    anzeigewert = anzeigewert * 10 + ziffer;

    System.out.println("anzeigewert ist " + anzeigewert
        + " am Ende von zifferGetippt.");
}
```

Listing 9.5

Zusätzliche Ausgabeanweisungen zur Fehlersuche.

```
/**
 * Gib die Werte aller Datenfelder dieses Objekts aus.
 * @param wo an welcher Stelle der Zustand ausgegeben wird
 */
private void zustandAusgeben(String wo)
{
    System.out.println("anzeigewert: " + anzeigewert
        + " linkerOperand: " + linkerOperand
        + " letzterOperator: " + letzterOperator
        + " am " + wo);
}
```

Listing 9.6

Eine Methode, die den Objektzustand ausgibt.

Wenn jede Methode der `Recheneinheit` eine Ausgabeanweisung zu Anfang stehen hat und am Ende einen Aufruf von `zustandAusgeben`, dann zeigt Abbildung 9.7 die Ausgabe, die sich durch einen Aufruf der Methode `testPlus` am Tester ergibt. (Diese Ausgabe wurde durch eine Version von `Recheneinheit` erzeugt, die im Projekt *Recheneinheit-mit-Ausgaben* zu finden ist.) Durch solche Ausgaben können wir uns ein Bild vom Kontrollfluss zwischen den Methoden machen. Beispielsweise können wir an der Reihenfolge der Zustandsausgaben erkennen, dass `plus` einen geschachtelten Aufruf von `letztenOperatorAnwenden` enthält.

```
clear wurde aufgerufen.
anzeigewert: 0 linkerOperand: 0 letzterOperator: am Ende von clear
zifferGetippt aufgerufen mit: 3
anzeigewert: 3 linkerOperand: 0 letzterOperator: am Ende von ziffer...
plus wurde aufgerufen.
letztenOperatorAnwenden wurde aufgerufen
anzeigewert: 3 linkerOperand: 3 letzterOperator: am Ende von letzte...
anzeigewert: 0 linkerOperand: 3 letzterOperator: + am Ende von plus
zifferGetippt aufgerufen mit: 4
anzeigewert: 4 linkerOperand: 3 letzterOperator: + am Ende von ziffer...
gleich wurde aufgerufen.
anzeigewert: 7 linkerOperand: 0 letzterOperator: + am Ende von gleich
```

Abbildung 9.7

Zusätzliche Ausgabeanweisungen zur Fehlersuche.

Ausgabeanweisungen können sehr nützlich sein zum Verstehen von Programmen oder bei der Suche nach Fehlern, aber sie haben auch eine Reihe von Nachteilen:

- Es ist nicht sehr praktikabel, jede Methode mit Ausgabeanweisungen zu versehen. Sie sind erst dann wirklich nützlich, wenn die richtigen Methoden angereichert werden.
- Zu viele Ausgabeanweisungen führen zu einer Informationsflut. Eine große Menge an Ausgaben kann es schwierig machen, die nützlichen zu erkennen. Insbesondere Ausgabeanweisungen in Schleifen sind ein Kandidat für dieses Problem.
- Sobald sie ihren Zweck erfüllt haben, kann es recht umständlich sein, sie wieder zu entfernen.
- Es kann auch vorkommen, dass sie wieder benötigt werden, obwohl sie gerade entfernt wurden. Es ist dann sehr frustrierend, sie erneut eingeben zu müssen!

Übung 9.31 Öffnen Sie das Projekt *Recheneinheit-mit-Ausgaben* und vervollständigen Sie das Einfügen von Ausgabeanweisungen in allen Methoden und dem Konstruktor.

Übung 9.32 Erzeugen Sie eine Instanz von **RecheneinheitTester** im Projekt und rufen Sie die Methode **allesTesten** auf. Helfen Ihnen die resultierenden Ausgaben bei der Suche nach der Ursache des Problems?

Übung 9.33 Haben Sie den Eindruck, dass die Ausgaben, die durch die angereicherte Klasse **Recheneinheit** erzeugt werden, zu knapp, zu umfangreich oder gerade angemessen sind? Wenn Sie den Eindruck haben, dass sie zu umfangreich oder zu knapp sind, dann fügen Sie entweder weitere Ausgabeanweisungen hinzu oder entfernen Sie unnötige Ausgaben.

Übung 9.34 Was sind jeweils die Vor- und die Nachteile von manuellen Ausführungen und von Ausgabeanweisungen bei der Fehlersuche und -beseitigung? Diskutieren Sie diese.

9.9.1 Ausgabeanweisungen an- und ausschalten

Wenn eine noch in der Entwicklung befindliche Klasse um Ausgabeanweisungen angereichert wird, dann sollen die Ausgaben nicht bei jeder Benutzung der Klasse erscheinen. Am besten ist, wenn wir die Ausgaben je nach Situation an- oder abschalten können. Üblicherweise wird dies durch ein zusätzliches boolesches Datenfeld für Testausgaben erreicht, von dessen Wert die Ausgaben abhängig gemacht werden. Listing 9.7 illustriert diese Idee.

Listing 9.7

Zusätzliche Ausgabeanweisungen zur Fehlersuche.

```
/**
 * Eine Zifferntaste wurde getippt.
 * @param ziffer die einzelne Ziffer
 */
public void zifferGetippt(int ziffer)
{
    if(testausgaben) {
        System.out.println("zifferGetippt aufgerufen mit: " + ziffer);
    }

    anzeigewert = anzeigewert * 10 + ziffer;
    if(testausgaben) {
        zustandAusgeben();
    }
}
```

Eine etwas ökonomischere Variante besteht darin, die direkten Ausgabeanweisungen durch Aufrufe einer der Klasse speziell hinzugefügten Ausgabemethode zu ersetzen.² Diese Ausgabemethode würde nur dann etwas ausgeben, wenn das Datenfeld **testausgaben** den Wert **true** hat. Auf diese Weise müssten die Aufrufe von Ausgabeanweisungen nicht in einer bedingten Anweisung stehen. Listing 9.8 verdeutlicht diesen Ansatz. Beachten Sie, dass diese Version annimmt, dass die Methode **zustandAusgeben** entweder selbst das boolesche Datenfeld abfragt oder die neue Methode **testausgabe** benutzt.

```
/**
 * Eine Zifferntaste wurde getippt.
 * @param ziffer die einzelne Ziffer
 */
public void zifferGetippt(int ziffer)
{
    testausgabe("zifferGetippt aufgerufen mit: " + ziffer);
    anzeigewert = anzeigewert * 10 + ziffer;
    zustandAusgeben();
}
```

```
/**
 * Gib die Information zur Fehlersuche nur aus, wenn
 * testausgaben true ist.
 * @param info die Information zur Fehlersuche
 */
public void testausgabe(String info)
{
    if(testausgaben) {
        System.out.println(info);
    }
}
```

Listing 9.8

Eine Methode zur Steuerung von Testausgaben.

Wie Sie anhand dieser Experimente sehen, benötigt man ein wenig Praxis, um herauszufinden, welches die ideale Detailtiefe für ein sinnvolles Ausgeben ist. In der Praxis werden Ausgabeanweisungen häufig nur einer Methode bzw. einer kleinen Anzahl an Methoden hinzugefügt, wenn man eine grobe Vorstellung davon hat, in welchem Bereich des Programms ein Fehler versteckt sein könnte.

9.10 Debugger

In Kapitel 3 haben wir einen Debugger benutzt, um uns eine bestehende Anwendung und die Interaktion ihrer Objekte zu erschließen. Auf sehr ähnliche Weise können wir einen Debugger auch benutzen, um Fehler zu finden.

Ein Debugger ist letztendlich ein Softwarewerkzeug, mit dem wir eine kontrollierte Ausführung eines Quelltextabschnitts durchführen können. Wir setzen typischerweise einen Haltepunkt an die Stelle, an der die Ausführung begonnen werden soll, und benutzen dann die Funktionen **SCHRITT** oder **SCHRITT HINEIN**, um eine Sequenz schrittweise zu durchlaufen.

Ein Vorteil dabei ist, dass der Debugger automatisch die Zustände der beteiligten Objekte verfolgt, und dies ist schneller und weniger fehlerträchtig als eine manuelle Ausführung. Ein Nachteil ist aber auch, dass ein Debugger kein Protokoll der

² Wir könnten diese Methode sogar in eine spezielle Klasse für die Unterstützung der Fehlersuche auslagern, aber wir wollen die Dinge in dieser Diskussion einfach halten.

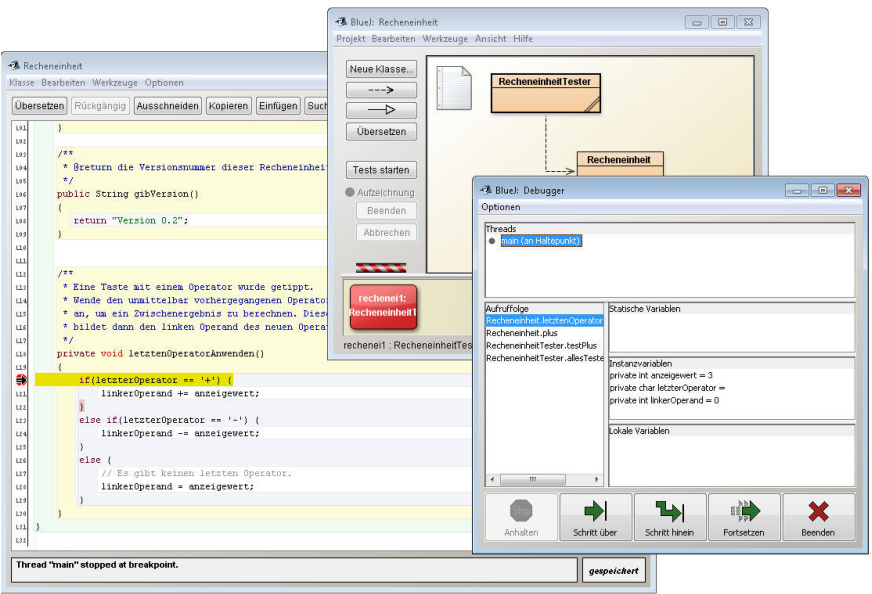
Objektzustände führt und man dadurch nicht ohne Weiteres einen früheren Objektzustand für einen Vergleich heranziehen kann.

Ein Debugger liefert üblicherweise auch Informationen über die *Aufrufsequenz*, auch *Stack* genannt, zu jedem Zeitpunkt. Die Aufrufsequenz zeigt den Namen der Methode, deren Anweisungen aktuell ausgeführt werden, den Namen der Methode, von der aus die aktuelle Methode aufgerufen wurde, den Namen der Methode, von der aus wiederum diese Methode aufgerufen wurde, und so weiter. Die Aufrufsequenz zeigt uns also ein Protokoll aller Methoden, die momentan aktiv und noch nicht beendet sind – ähnlich unserem Vorgehen bei der manuellen Ausführung, als wir Markierungen an die Aufrufanweisungen gemacht haben.

In BlueJ wird die Aufrufsequenz im linken Bereich des Debugger-Fensters angezeigt (Abbildung 9.8). Jeder Methodenname in dieser Liste kann selektiert werden, um die lokalen Variablen der jeweiligen Methode anzeigen zu lassen.

Abbildung 9.8

Das Debugger-Fenster von BlueJ, bei dem die Ausführung bei einem Haltepunkt gestoppt wird.



Übung 9.35 Öffnen Sie das Projekt *Rechenheit*, setzen Sie einen Haltepunkt in der ersten Zeile der Methode `testPlus` der Klasse *RechenheitTester*. Führen Sie diese Methode aus. Wenn der Debugger erscheint, gehen Sie Schritt für Schritt durch den Quelltext. Experimentieren Sie sowohl mit den Knöpfen **SCHRITT ÜBER** sowie **SCHRITT HINEIN**.

Übung 9.36 Zusatzaufgabe. In der Praxis könnten Sie zu dem Entschluss kommen, dass die Version der *Rechenheit* von Hacker T. Largebrain zu fehlerhaft ist, um repariert zu werden. Schreiben Sie also stattdessen eine völlig neue Implementierung. Das Projekt *Rechner-GUI* enthält Klassen, die die grafische Benutzungsschnittstelle aus Abbildung 9.6 anbieten. Sie können dieses Projekt als Basis für Ihre eigene Implementierung der Klasse *Rechenheit* nutzen. Dokumentieren Sie Ihre neue Klasse sorgfältig und implementieren Sie

eine Testklasse mit gründlich durchdachten Testfällen. Die Erfahrungen, die Sie mit dem Quelltext von Hacker gemacht haben, sollten Ihrem Nachfolger erspart bleiben! Achten Sie darauf, zum Testen eine spezielle Modultest-Klasse zu verwenden, anstatt Tests in eine Standardklasse zu schreiben. Wie Sie gesehen haben, macht dies das Zusichern von korrekten Ergebnissen viel leichter.

9.11 Das Debugging von Streams (fortgeschritten)

Alle Debugging-Techniken, die wir in diesem Kapitel beschrieben haben, lassen sich ebenso gut auf Code anwenden, der Streams verwendet. Der einzige kleine Unterschied ist die Komplexität der Anweisungen, die Streams benutzen. Es ist nicht ungewöhnlich, dass eine einzelne Anweisung drei oder mehr Operationen innerhalb einer Pipeline hintereinanderschaltet, und die Reihenfolge und der logische Fluss zwischen diesen Operationen sind entscheidend für das korrekte Ergebnis der Anweisung. Daher ist es oft eine gute Idee, den Zustand eines Streams an Zwischenpunkten der Pipeline zu untersuchen.

Die Methode **peek** ist eine Stream-Operation, die äquivalent zur Benutzung von **print**-Anweisungen ist und es ermöglicht, den Zustand einer Pipeline an verschiedenen Punkten zu untersuchen. Es ist eine intermediäre Operation, die an jedem beliebigen Punkt innerhalb der Pipeline eingefügt werden kann. Als Parameter bekommt **peek** einen **Consumer**-Lambda-Ausdruck³, reicht aber auch die Elemente ihres Eingabe-Streams unverändert an ihren Ausgabe-Stream durch.

Nehmen wir beispielsweise an, Sie überarbeiten im Projekt *Tiermonitor* einen einigermaßen komplexen Code – der von jemand anderem geschrieben wurde. Der Code soll eine Liste von Sichtungen für ein bestimmtes Tier, ein Gebiet und einen Melder erzeugen. Möglicherweise wurde dies folgendermaßen in einer einzigen Anweisung untergebracht:

```
List<Sichtung> ergebnis =
    sichtungen.stream()
        .filter(record -> tier == datensatz.gibTier() &&
            gebiet == datensatz.gibGebiet() &&
            melder == datensatz.gibMelder())
        .collect(Collectors.toList());
```

Stellen Sie sich vor, dass die Liste, die dieser Code erzeugt, immer leer ist, auch wenn Sie Testdaten mit Sichtungsdatensätzen verwenden, die definitiv übereinstimmen müssten. Offensichtlich stimmt irgendetwas mit der **filter**-Operation nicht, aber Sie können das Problem nicht direkt ausfindig machen. Einer der leichtesten Wege, der Sache auf den Grund zu gehen, wäre es, das Filtern in drei separate Operationen aufzuspalten und die Datenfelder der verbleibenden Elemente nach jedem Filterschritt auszugeben. Die folgende Version macht genau dieses:

³ Zum Typ **Consumer**, siehe Abschnitt 12.6.7.

```
List<Sichtung> ergebnis =
    sichtungen.stream()
        .filter(datensatz -> tier == datensatz.gibTier())
        .peek(r -> System.out.println(r.gibTier()))
        .filter(datensatz -> gebiet == datensatz.gibGebiet())
        .peek(r -> System.out.println(r.gibGebiet()))
        .filter(datensatz -> melder == datensatz.gibMelder())
        .peek(r -> System.out.println(r.gibDetails()))
        .collect(Collectors.toList());
```

Mit dieser Version werden Sie recht wahrscheinlich herausfinden, dass der erste Filterschritt überhaupt keine Datensätze an die nächste Operation weitergibt, weil der Test auf textuelle Übereinstimmung der Zeichenketten durch die **equals**-Methode nicht durchgeführt wurde. Sobald dies behoben ist, können die Tests erneut ausgeführt werden, dann kann die **peek**-Methode entfernt werden.

Eine **peek**-Operation eignet sich auch, um eine Position in der Mitte einer Pipeline-Folge als Haltepunkt für einen Debugger zu setzen. In diesem Fall sind Sie vermutlich eher an den Zuständen der Objekte interessiert, anstatt daran etwas auszugeben. Daher würde man hier einen **Consumer**-Lambda-Ausdruck, der nichts macht, als Parameter für **peek** benutzen, zum Beispiel:

```
peek(r -> { })
```

9.12 Die Wahl der richtigen Teststrategie

Wie Sie sehen konnten, gibt es verschiedene Teststrategien: die handschriftliche oder mündliche Ausführung, das Einfügen von Ausgabeanweisungen (temporär oder permanent und durch Schalter gesteuert), das interaktive Testen mithilfe der Objektleiste, das Schreiben eigener Testklassen oder den Einsatz einer speziellen Modultest-Klasse.

In der täglichen Praxis verwenden wir je nach Situation unterschiedliche Teststrategien. Manuelle Ausführung, Ausgabeanweisungen und interaktives Testen eignen sich insbesondere für das Testen von neu geschriebenen Quelltexten, das Analysieren der Funktionsweise eines Programmabschnitts oder für die Fehlersuche. Der Vorzug dieser Techniken ist, dass sie schnell und leicht einzusetzen sind, in jeder Programmiersprache eingesetzt werden können und (mit Ausnahme des interaktiven Testens) nicht von der verwendeten Umgebung abhängig sind. Ihr größter Nachteil ist, dass sie nicht leicht zu wiederholen sind. Das ist für die Fehlersuche in Ordnung, aber zum Testen benötigen wir etwas Besseres: Wir benötigen einen Mechanismus, der uns erlaubt, die Regressionstests schnell zu wiederholen. Die Verwendung von Modultest-Klassen hat dagegen den Vorteil, dass die einmal eingerichteten Tests beliebig oft wiederholt werden können.

Der von Hacker eingeschlagene Weg, seine eigenen Testklassen zu schreiben, war also ein Schritt in die richtige Richtung. Er war nur zu kurz gegriffen. Denn wie wir mittlerweile wissen, genügt es nicht, wenn die Klassen ausreichend Methodenaufrufe zum Testen enthalten, die Klassen müssen die Ergebnisse der Methode auch auswerten und erkennen, wenn ein Test gescheitert ist. Durch Einsatz einer passenden Modultest-Klasse hätte dieses Problem besser und leichter gelöst werden können.

Übung 9.37 Öffnen Sie Ihr Projekt erneut und verbessern Sie die Testunterstützung, indem Sie die Testklasse von Hacker durch eine Modultest-Klasse ersetzen, die **Recheneinheit** zugeordnet ist. Richten Sie ähnliche Tests ein, wie Hacker sie nutzte, fügen Sie gegebenenfalls weitere Tests hinzu, die Sie für sinnvoll erachten, und achten Sie auf korrekte Zusicherungen.

9.13 Techniken umsetzen

Die in diesem Kapitel vorgestellten Techniken können benutzt werden, um ein neues Programm zu verstehen oder um es auf Fehler zu untersuchen. Das Projekt *Ziegelsteine* bietet Ihnen die Möglichkeit, diese Techniken in einem neuen Szenario auszuprobieren. Das Projekt enthält Teile einer Anwendung für eine Firma, die Ziegelsteine herstellt. Ziegelsteine werden an Kunden auf Paletten (als Stapel von Ziegeln) ausgeliefert. Die Klasse **Palette** bietet Methoden, mit denen die Höhe und das Gewicht einer Palette in Abhängigkeit von der Anzahl der Ziegel auf der Palette abgefragt werden können.

Übung 9.38 Öffnen Sie das Projekt *Ziegelsteine*. Testen Sie es. Es gibt mindestens vier Fehler in diesem Projekt. Versuchen Sie, diese zu finden und zu beheben. Welche Techniken haben Sie zur Fehlersuche angewendet? Welche Technik war am hilfreichsten?